Neural networks for classification

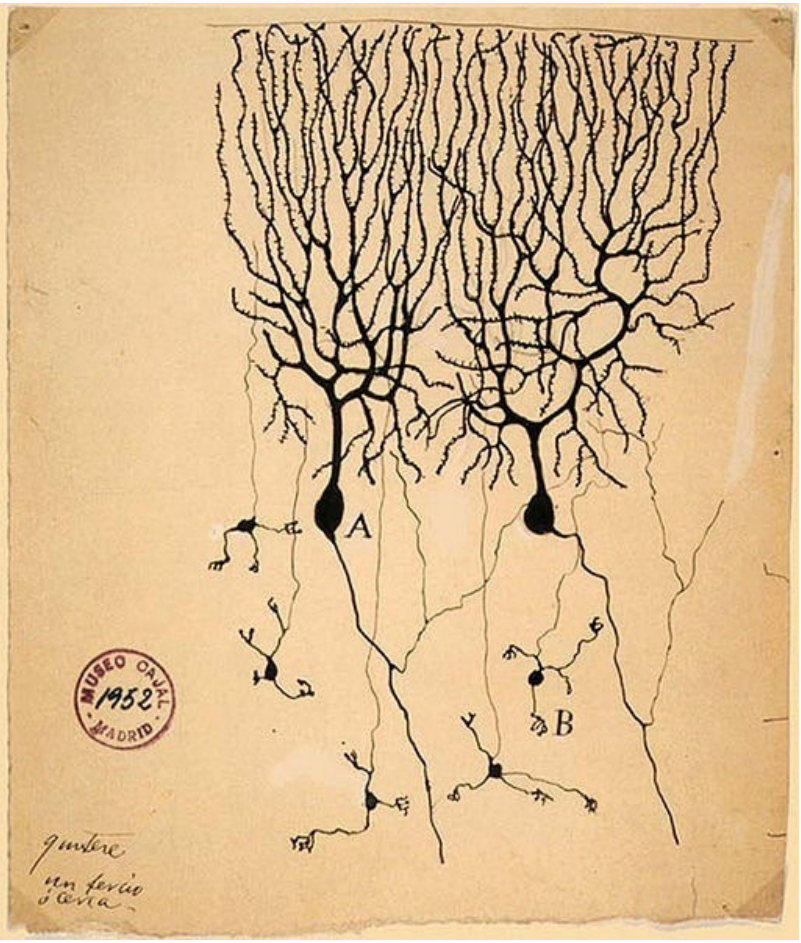
alberto.suarez@uam.es

Escuela Politécnica Superior

The brain

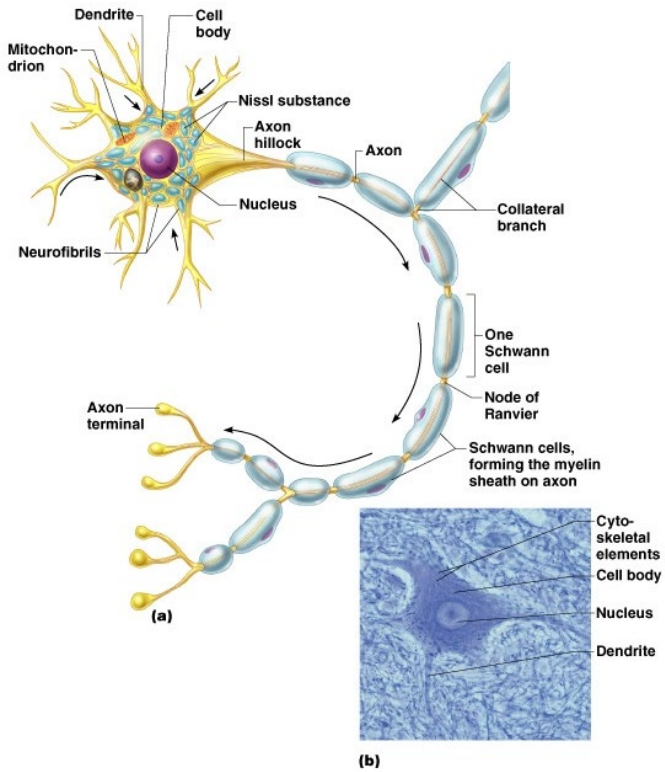
Connected network of **neurons** that communicate by means of **electric** and **chemical** signals

- ~ 10^{11} neurons
- ~ 1000 synapses per neuron



Cajal's drawing of neurons in cerebellar cortex of a pigeon. The letter A marks the Purkinje cells with their characteristic dendritic ramifications.
[Santiago Ramón y Cajal, 1899.
Instituto Santiago Ramón y Cajal, Madrid, Spain.]

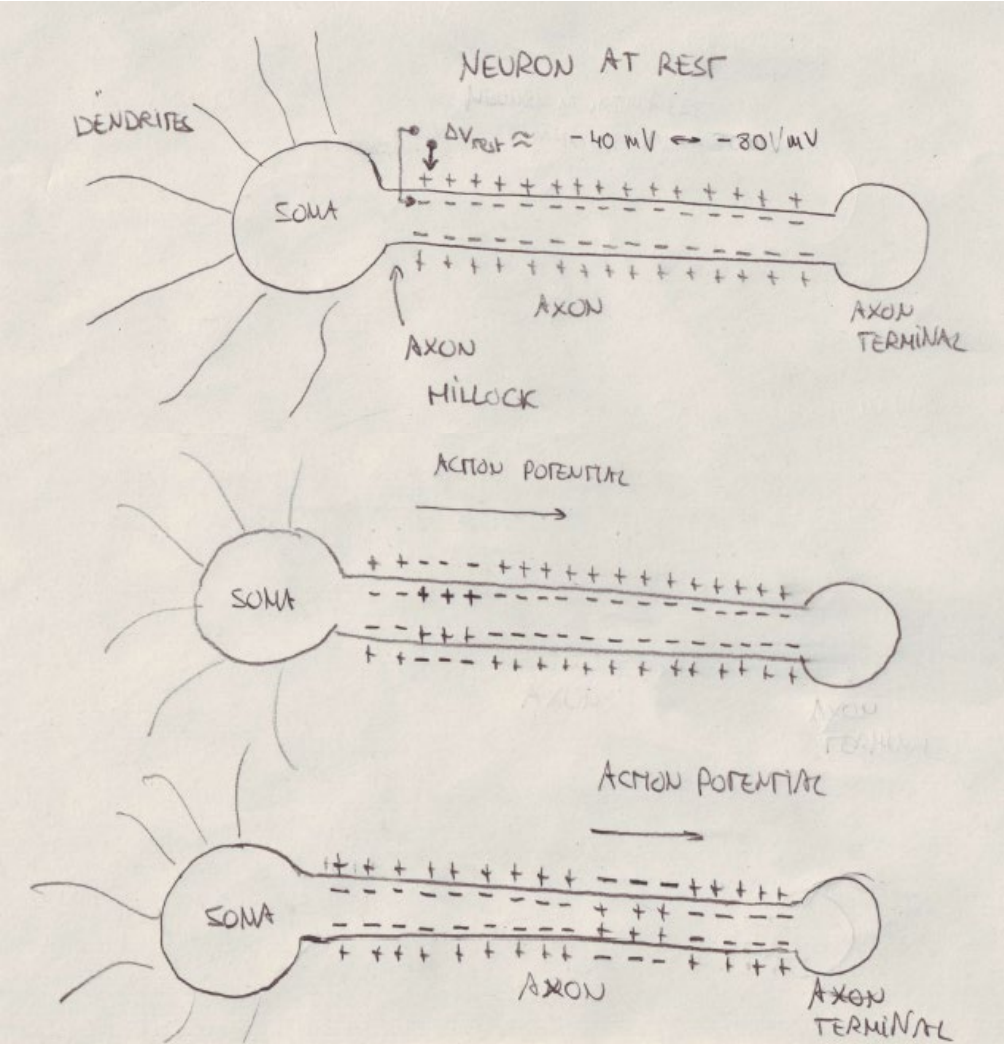
A biological neuron

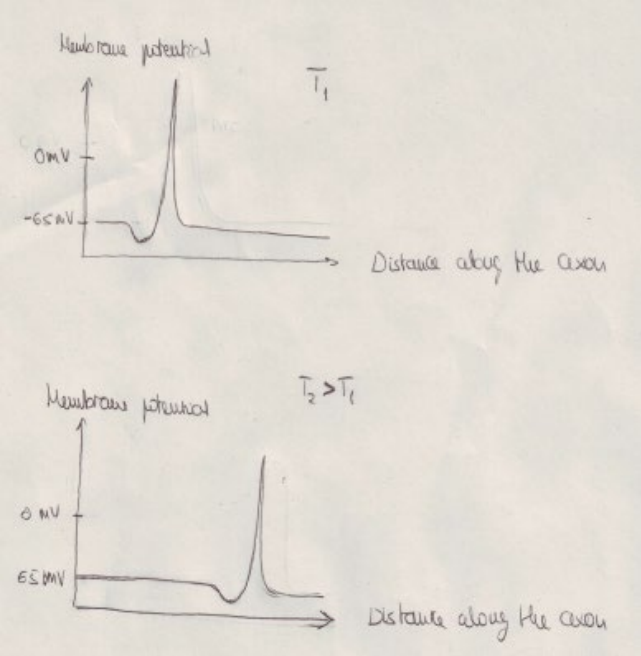


Copyright © 2006 Pearson Education, Inc., publishing as Benjamin Cummings.

2-Minute Neuroscience: Synaptic Transmission
Neuroscientifically Challenged
https://youtu.be/6qS83wD29PY

Neurons at rest / firing

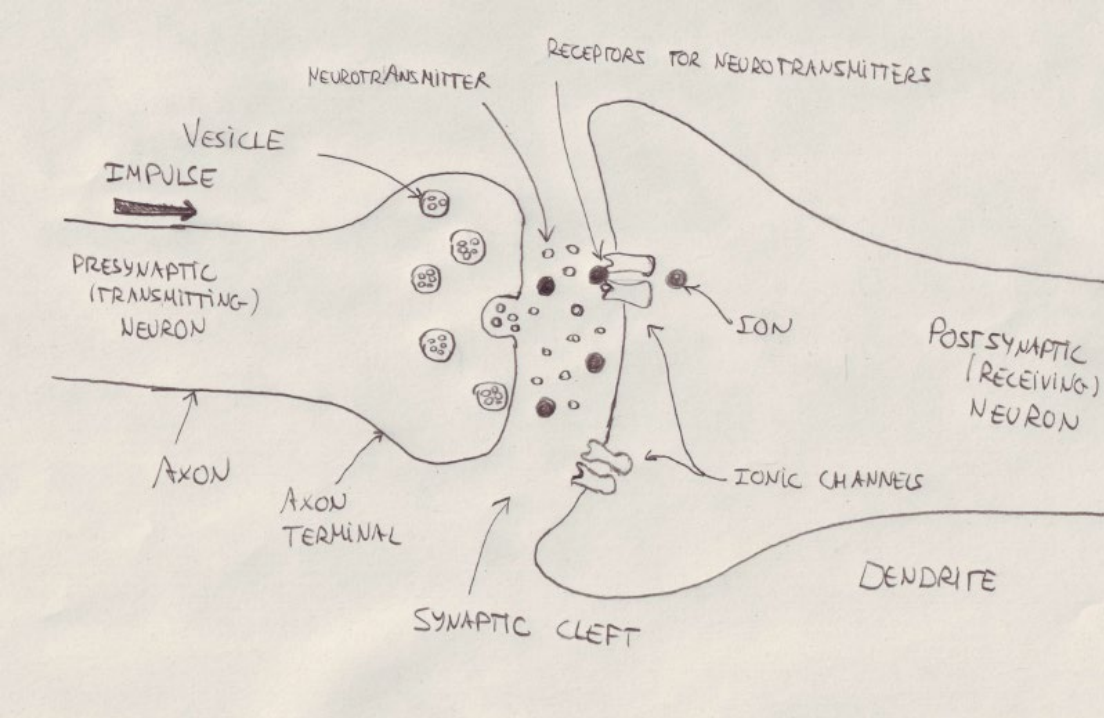




Communication among neurons

- **Communication among neurons** takes place when the **action potential** arrives to the **axon terminal** of the presynaptic neuron, causing the influx of calcium ions (Ca^{++}) .
- This **ion influx** initiates the release of **neurotransmitters** than **transport the signal** across the **synaptic cleft** to the postsynaptic neuron.
- Neurotransmitters can be either **inhibitory** or **excitatory**, and either block or stimulate the generation of an action potential in the postsynaptic neuron.
- The response of the postsynaptic neuron depends of which signals (inhibitory or excitatory) dominate.

The synapse



2-Minute Neuroscience: Synaptic Transmission
Neuroscientifically Challenged
https://youtu.be/WhowH0kb7n0

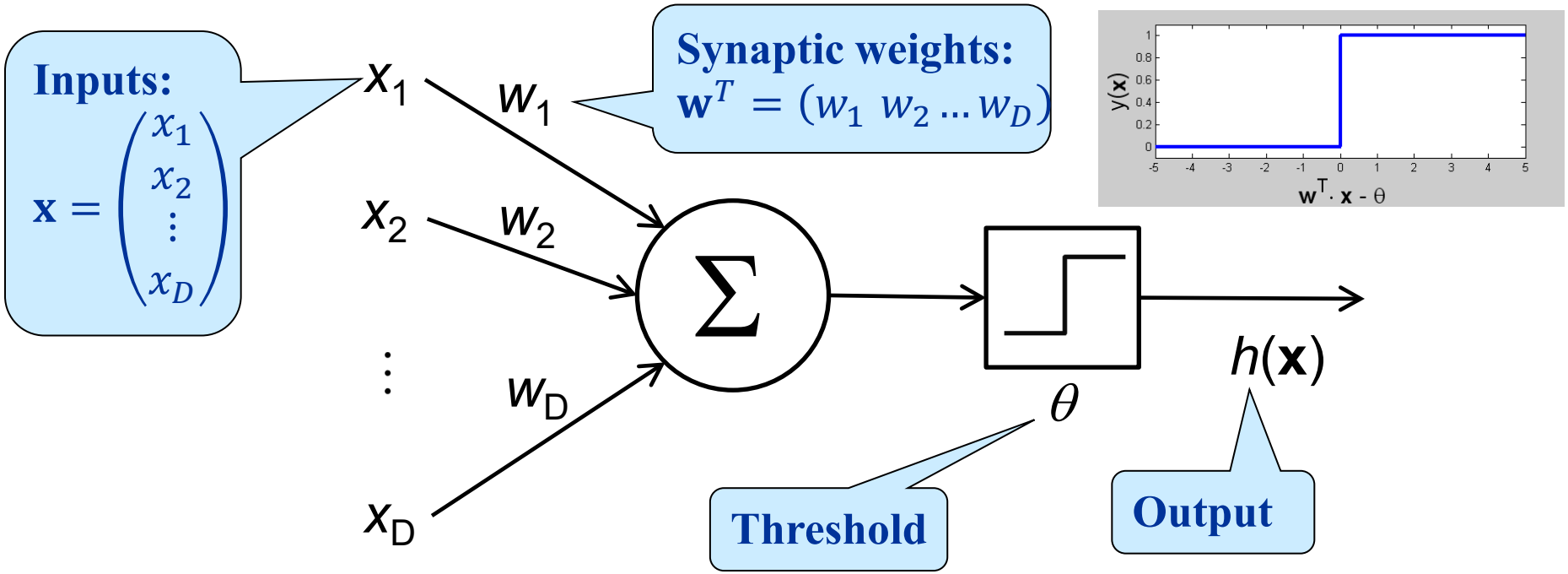
Stylized facts about biological neurons

- A **neuron receives impulses** (both **excitatory** and **inhibitory**) of different strength **from many other neurons**.
- The **neuron** then **integrates (sums) the impulses** received over space and time.
- If the resulting **integrated signal is above a threshold**, the neuron generates an action potential (**the neuron "fires"**).
- The **response of the neuron** is of the type "**all-or-nothing**"
- The **action potential,** generated at the axon hillock, is **transported along the axon** until the axon terminal.
- The signal is then transmitted to the following neuron in the network through a **synapse** by means of neurotransmitters.
- The **strength of the synapse can change (learning)**.

Lessons for pattern recognition

- Importance of **connectionism**: units with simple procession that interact in a complex network.
- **Distributed & sub-symbolic representation** of knowledge.
- **Parallelism.**
- **Thresholding mechanism** for robust classification.
- **A mechanism for learning**: learn by adjusting the synaptic weights.

An artificial neuron



$$h(\mathbf{x}) = \mathbb{I}[\mathbf{w}^T\mathbf{x} > \theta] = \begin{cases} 1 & \text{if} \quad w_1x_1 + w_2x_2 + \cdots + w_Dx_D > \theta \\ 0 & \text{if} \quad w_1x_1 + w_2x_2 + \cdots + w_Dx_D < \theta \end{cases}$$

McCulloch, W. S., and Pitts, W. "A Logical Calculus of Ideas Immanent in Nervous Activity." Bulletin of mathematical biophysics, 5, pp. 115-133 (1943). Reprinted in McCulloch, W. S., *Embodiments of mind*. Cambridge, MA: MIT Press.

AND gate data

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \qquad h(\mathbf{x}) = \begin{cases} 1 & \text{if} \quad x_1 + x_2 \geq 1.5 \\ 0 & \text{if} \quad x_1 + x_2 < 1.5 \end{cases}$$

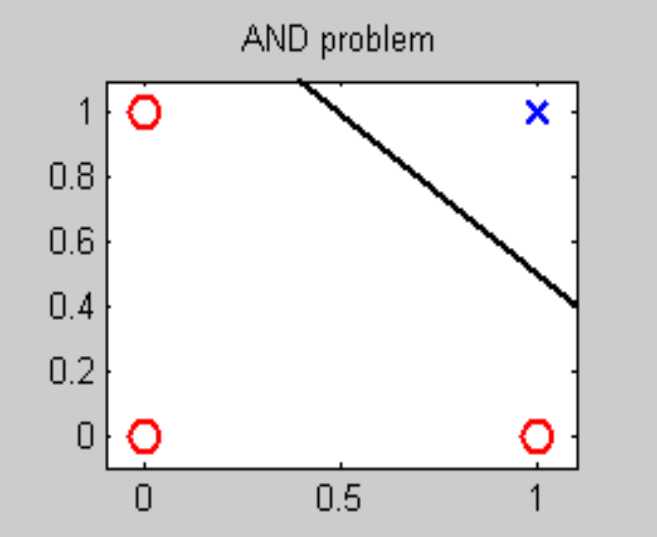
Synaptic weights:

$$\mathbf{w} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Threshold:

$$\theta = 1.5$$

x_1	x_2	$w_1x_1 + w_2x_2$	h
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2	1



OR gate data

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \qquad h(\mathbf{x}) = \begin{cases} 1 & \text{if} \quad x_1 + x_2 \geq 0.5 \\ 0 & \text{if} \quad x_1 + x_2 < 0.5 \end{cases}$$

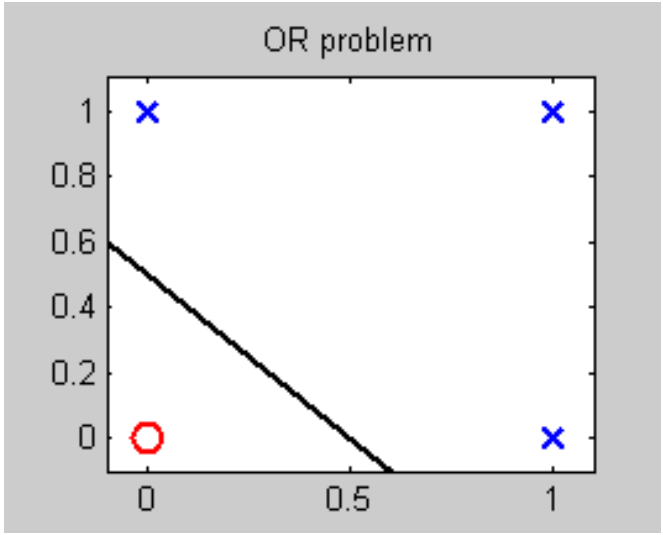
Synaptic weights:

$$\mathbf{w} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Threshold:

$$\theta = 0.5$$

x_1	x_2	$w_1x_1 + w_2x_2$	h
0	0	0	0
0	1	1	1
1	0	1	1
1	1	2	1



OR gate with noisy inputs

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \qquad h(\mathbf{x}) = \begin{cases} 1 & \text{if} \quad x_1 + x_2 \geq 0.5 \\ 0 & \text{if} \quad x_1 + x_2 < 0.5 \end{cases}$$

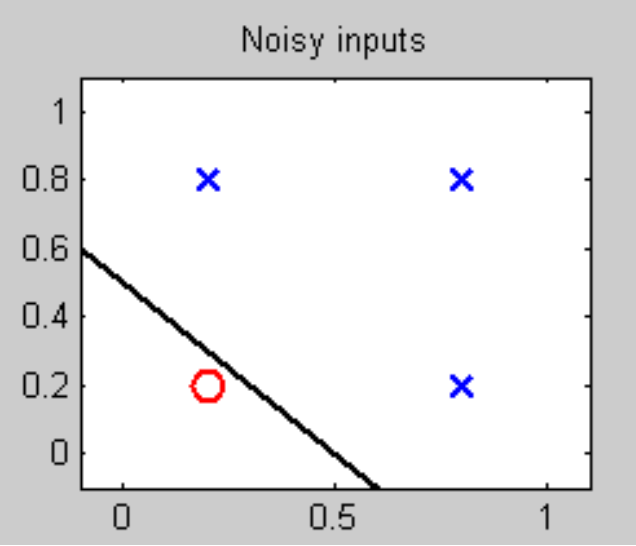
Synaptic weights:

$$\mathbf{w} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Threshold:

$$\theta = 0.5$$

x_1	x_2	$w_1x_1 + w_2x_2$	h
0.2	0.2	0.4	0
0.2	0.8	1	1
0.8	0.2	1	1
0.8	0.8	1.6	1



OR gate with noisy network parameters

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \qquad h(\mathbf{x}) = \begin{cases} 1 & \text{si} \quad 0.8\,x_1 + 0.7x_2 \geq 0.6 \\ 0 & \text{si} \quad 0.8\,x_1 + 0.7x_2 < 0.6 \end{cases}$$

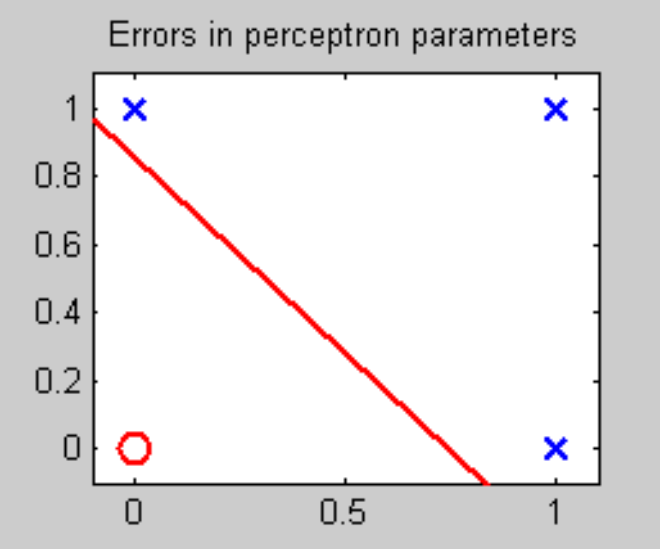
Synaptic weights:

$$\mathbf{w} = \begin{pmatrix} 0.8 \\ 0.7 \end{pmatrix}$$

Threshold:

$$\theta = 0.6$$

x_1	x_2	$w_1x_1 + w_2x_2$	h
0	0	0	0
0	1	0.7	1
1	0	0.8	1
1	1	1.5	1



OR gate with noisy inputs & parameters

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \qquad h(\mathbf{x}) = \begin{cases} 1 & \text{si} \quad 0.8\,x_1 + 0.7x_2 \geq 0.6 \\ 0 & \text{si} \quad 0.8\,x_1 + 0.7x_2 < 0.6 \end{cases}$$

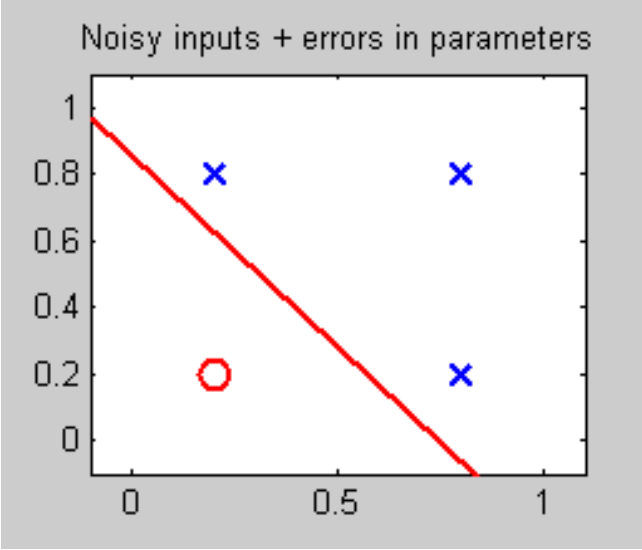
Synaptic weights:

$$\mathbf{w} = \begin{pmatrix} 0.8 \\ 0.7 \end{pmatrix}$$

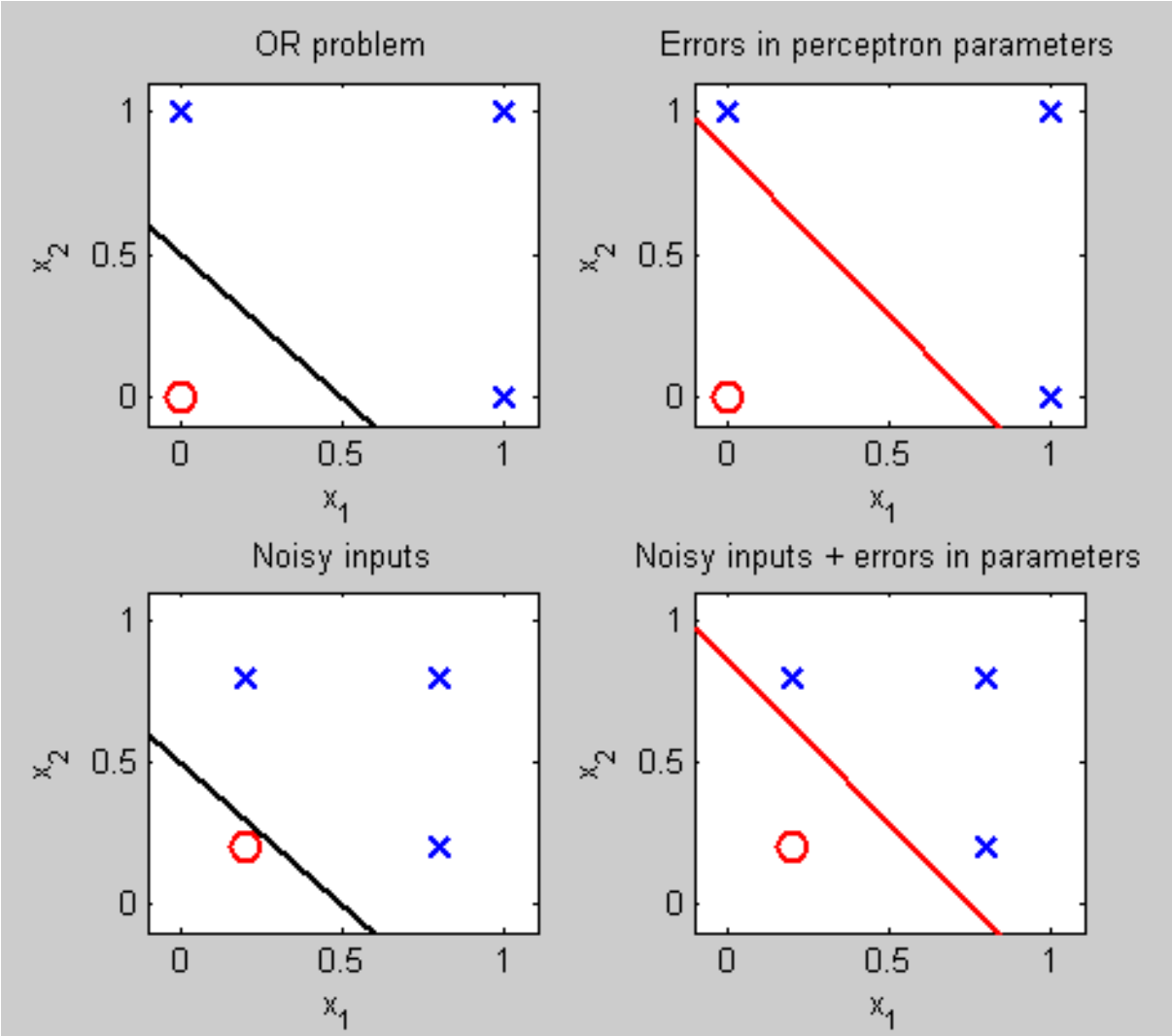
Threshold:

$$\theta = 0.6$$

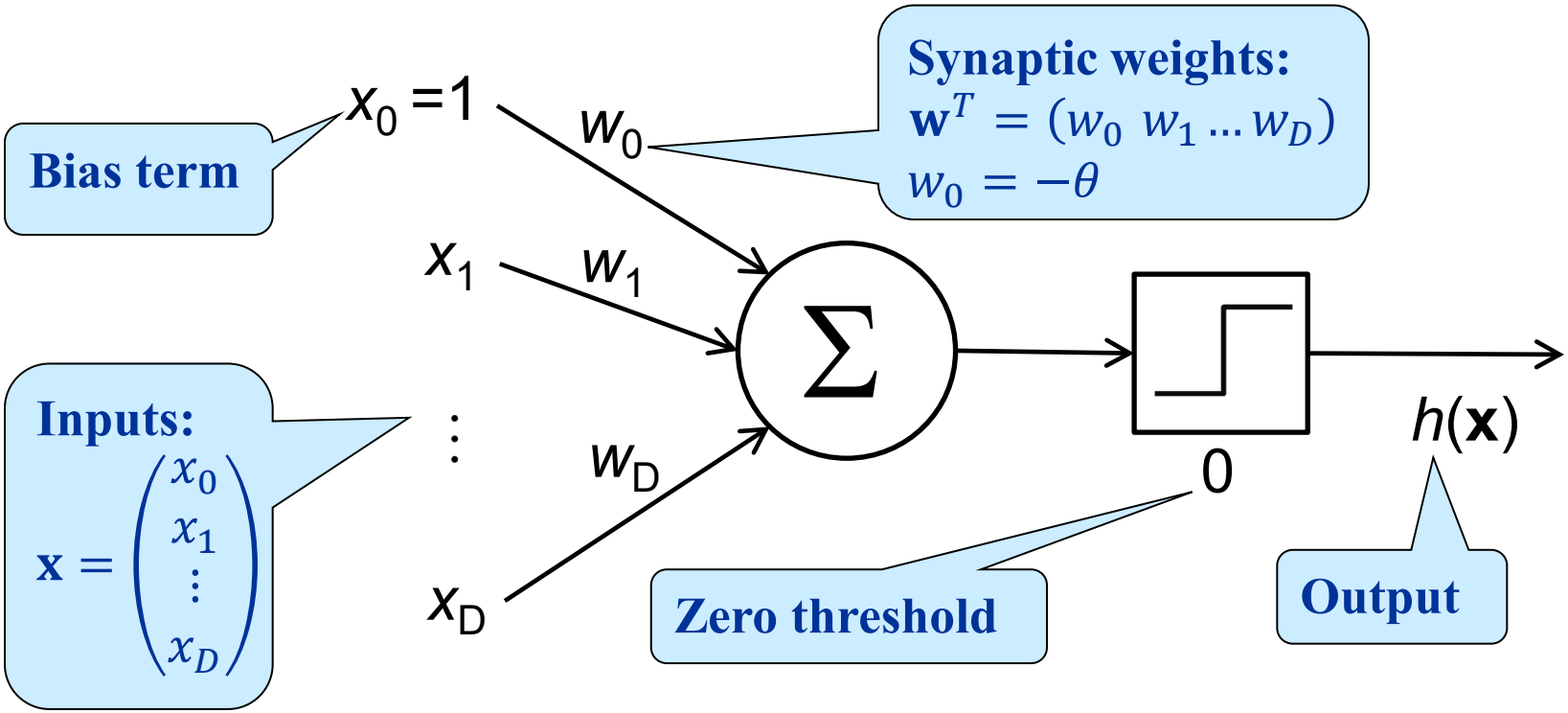
x_1	x_2	$w_1x_1 + w_2x_2$	h
0.2	0.2	0.30	0
0.2	0.8	0.72	1
0.8	0.2	0.78	1
0.8	0.8	1.20	1



OR gate: Summary



The bias term



$$h(\mathbf{x}) = \mathbb{I}[\mathbf{w}^T\mathbf{x} > 0] = \begin{cases} 1 & \text{if } w_0x_0 + w_1x_1 + \cdots + w_Dx_D > 0 \\ 0 & \text{if } w_0x_0 + w_1x_1 + \cdots + w_Dx_D < 0 \end{cases}$$

Indicator (Heavyside step function)

Rosenblatt perceptron

- **Labeled instances:** $\mathcal{D}_{train} = \{(\mathbf{x}_n, c_n)\}_{n=1}^{N_{train}}$
  - **Attributes:** $\mathbf{x}_n \in \mathcal{X}$
  - **Class labels:** $c_n \in \{C_1, C_2\}$
  - **Class label encoding:** $t_n = \begin{cases} 1 & \text{if} \quad c_n = C_1 \\ 0 & \text{if} \quad c_n = C_2 \end{cases}$
- **Classification:**
  - **Given the vector of attributes:** $\mathbf{x}_n \in \mathcal{X}$
  - Predict

$$h(\mathbf{x}_n) = \mathbb{I}[\mathbf{w}^T\mathbf{x}_n > 0] = \begin{cases} 1 & \text{if} \quad \mathbf{w}^T\mathbf{x}_n > 0 \\ 0 & \text{if} \quad \mathbf{w}^T\mathbf{x}_n < 0 \end{cases}$$

Classify as C_1

Classify as C_2

$\mathbf{x}_n$ is correctly classified if $h(\mathbf{x}_n) = t_n$

Geometric interpretation

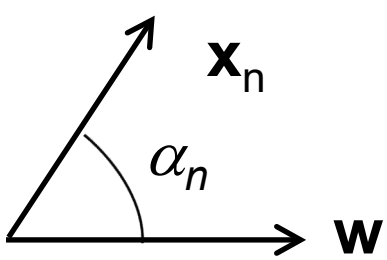
- **Vector of attributes:** $\mathbf{x}_n \in \mathcal{X}$
- **Vector of weights:** $\mathbf{w} \in \mathbb{R}^{D+1}$

$$\alpha_n = \text{angle}(\mathbf{x}_n, \mathbf{w})$$

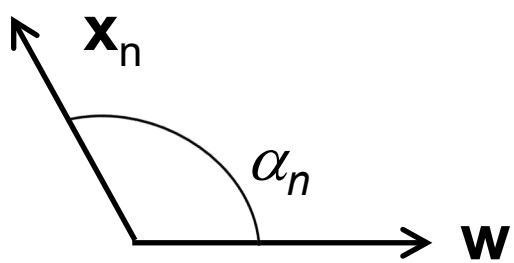
Classify as C_1

$$h(\mathbf{x}_n) = \mathbb{I}[\mathbf{w}^T\mathbf{x}_n > 0] = \mathbb{I}[\cos\alpha_n > 0] = \begin{cases} 1 & \text{if} \quad -\frac{\pi}{2} < \alpha_n < \frac{\pi}{2} \\ 0 & \text{if} \quad \frac{\pi}{2} < \alpha_n < \frac{3\pi}{2} \end{cases}$$

Classify as C_2



Classified as C_1



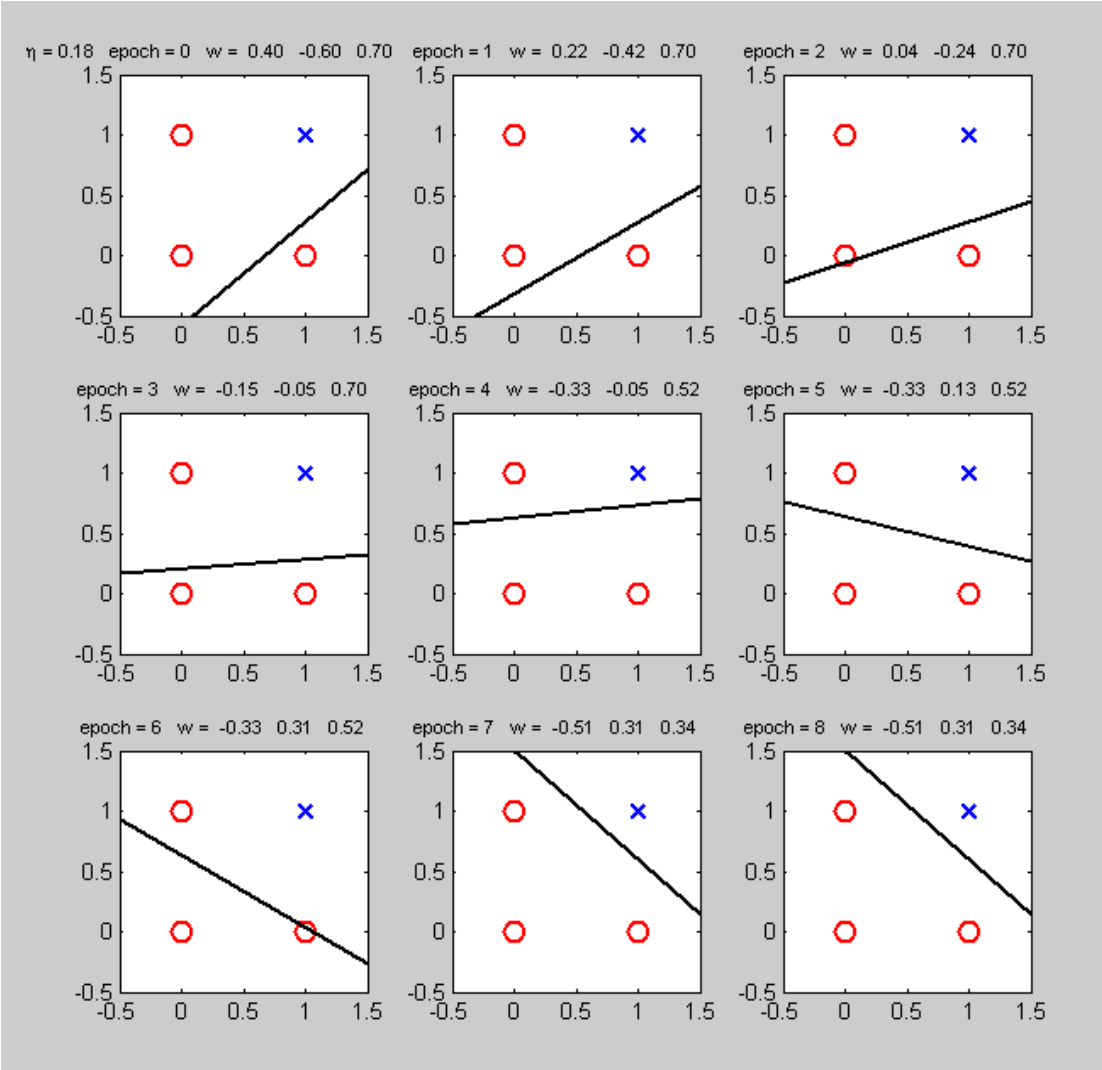
Classified as C_2

Rosenblatt perceptron: Learning

- **Input**: Training data: $\mathcal{D}_{train} = \{(\mathbf{x}_n, t_n)\}_{n=1}^{N_{train}}$
  Learning rate: $\eta > 0$
- **Output:** Perceptron weights $\mathbf{w}$
- **Code:** 1. $\mathbf{w} \sim \mathrm{U}[-0.5, 0.5]^{D+1}$ # Randomly initialize weights
  2. $k = 0$ # Initialize epoch counter
  3. While convergence criteria not met [**epoch loop**]
     3.1 $k = k + 1$ # Increment epoch counter
     3.2 For $n = 1, \ldots, N_{train}$ # Loop through instances
     $h(\mathbf{x}_n) = \mathbb{I}[\mathbf{w}^T \mathbf{x}_n > 0]$ # Classify
     $\delta_n = h(\mathbf{x}_n) - t_n$ # signed error
     $\mathbf{w} = \mathbf{w} - \eta \delta_n \mathbf{x}_n$ # Update weights

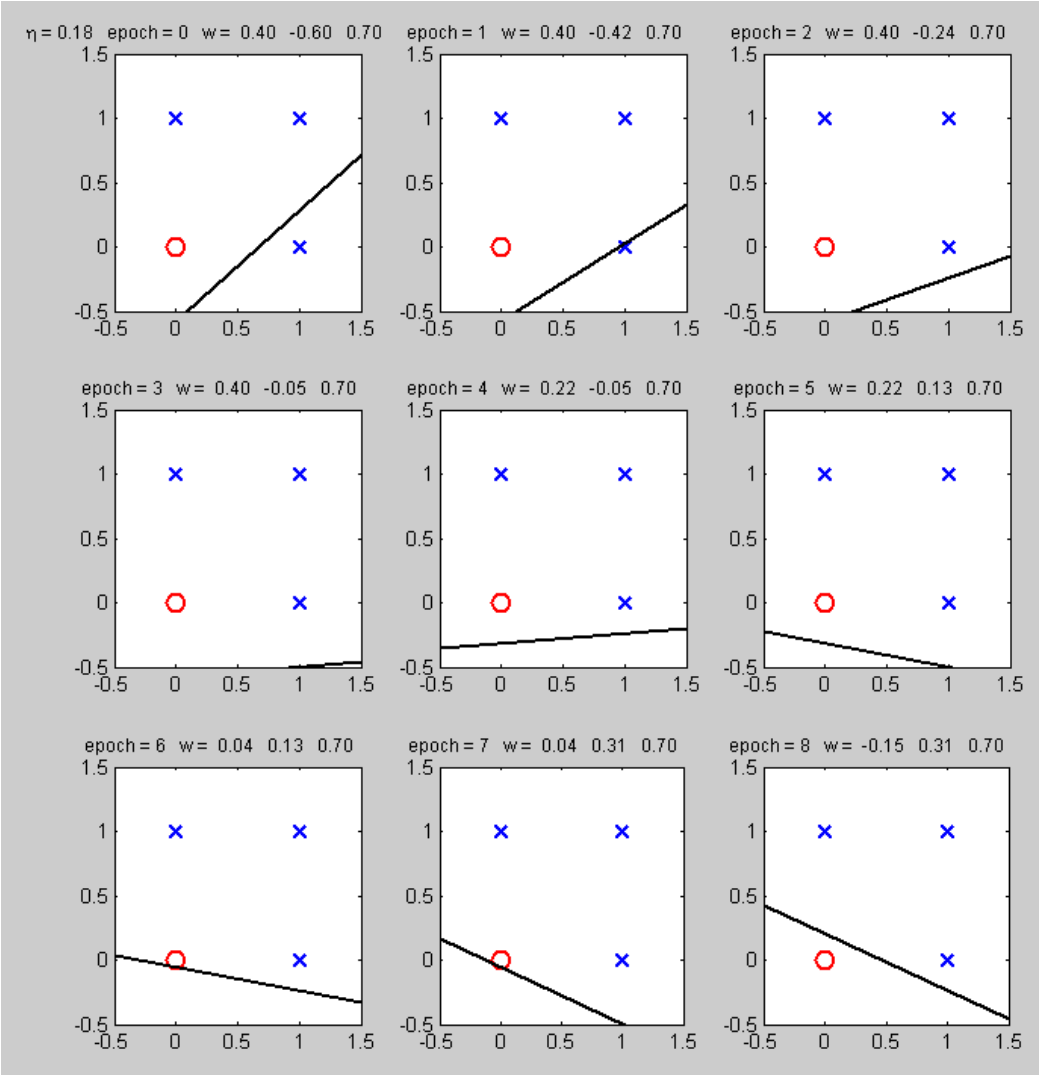
Learning the AND gate data (Rosenblatt)

x_1	x_2	h
0	0	0
0	1	0
1	0	0
1	1	1



Learning the OR gate data (Rosenblatt)

x_1	x_2	h
0	0	0
0	1	1
1	0	1
1	1	1



Rosenblatt perceptron: objective function

- **Rosenblatt criterion**:

$$J(\mathbf{w}) = \sum_{n=1}^{N_{train}} \delta_n \, \mathbf{w}^T \mathbf{x}_n$$

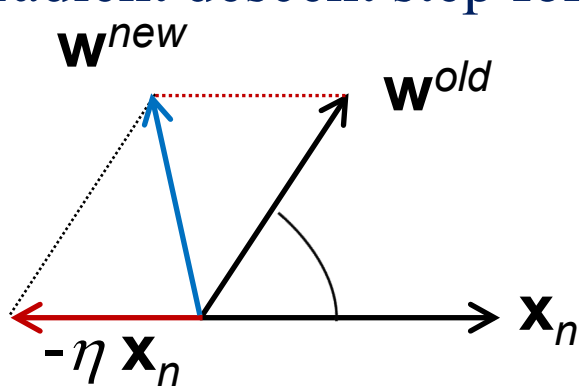
$$\delta_n = h(\mathbf{x}_n; \mathbf{w}) - t_n$$
$$= \mathbb{I}[\mathbf{w}^T \mathbf{x}_n > 0] - t_n$$

- **Learning** = Minimize Rosenblatt criterion

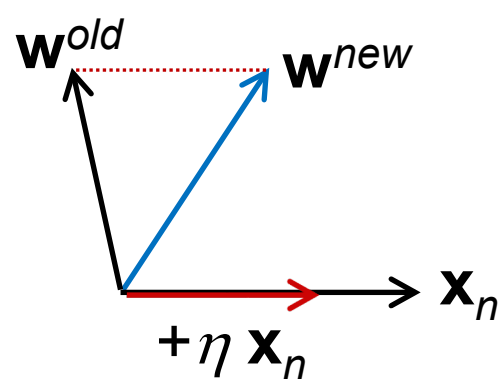
$$\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} = \sum_{n=1}^{N_{train}} \delta_n \, \mathbf{x}_n$$

$\frac{\partial \delta_n}{\partial \mathbf{w}} = 0$ everywhere except at $\mathbf{w}^T \mathbf{x}_n = 0$

- Gradient descent step for **online learning**: $\mathbf{w} = \mathbf{w} - \eta \delta_n \mathbf{x}_n$



$h(x_n) = 1$, but $t_n = 0 \rightarrow \delta_n = +1$

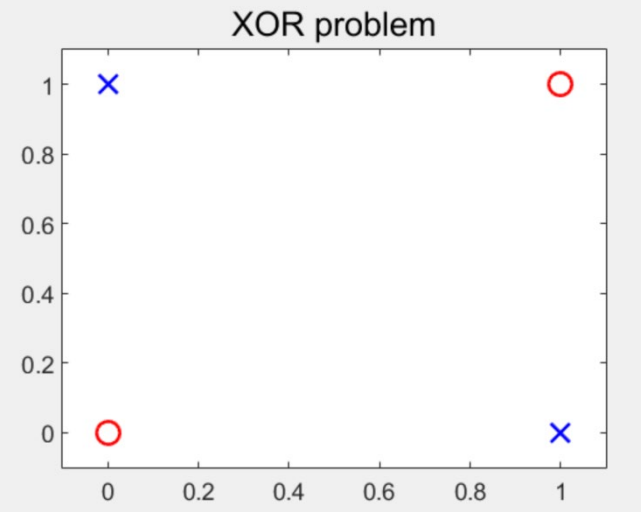


$h(x_n) = 0$, but $t_n = 1 \rightarrow \delta_n = -1$

XOR: A non-linearly separable problem

- The single-layer perceptron can only address problems that are linearly separable
- Therefore, the simple XOR problem, which is not linearly separable, cannot be solved using this learning machine.

x_1	x_2	h
0	0	0
0	1	1
1	0	1
1	1	0



We need a non-linear classifier!

Supervised learning: classification

Machine learning by **induction from labeled data**

$$\mathcal{D} = \{(\mathbf{x}_n, c_n)\}_{n=1}^{N}$$

$\mathbf{x} \in \mathcal{X}$: vector of **attributes**, features, independent variables, input variables. covariates…

Bias term $x_{n0} = 1$

$\mathbf{x}_n^T = (x_{n0}\ x_{n1}\ \ x_{n2}\ \dots\ x_{nD})$ [$(D+1)$-dimensional vector]

$c \in \mathcal{C}$: (class) **label**, dependent variable, outcome, target, …

Classification: **Labels** c are **discrete** $(e.g.\ c \in \{C_1, \dots, C_K\})$

Learning algorithm: $\mathcal{L}$

$\mathbf{w}$: Model parameters

$\mathcal{L}$: $\mathcal{D} = \{(\mathbf{x}_n, c_n)\}_{n=1}^{N} \rightarrow h(\,\cdot\,; \mathbf{w})$

$h(\,\cdot\,; \mathbf{w})$: $\mathbf{x} \in \mathcal{X} \rightarrow h(\mathbf{x}; \mathbf{w}) \in \{C_1, \dots, C_K\}$

Binary classification

n	x_{n1}	x_{n2}	...	x_{nD}	y_n
1	2.3	0	...	10.3	C_0
2	2.5	1	...	13.1	C_1
3	2.6	0	...	-2.7	C_1
4	2.7	-1	...	-5.4	C_0
5	2.9	0	...	2.1	C_1
6	3.1	0	...	-10.9	C_0

Binary classification

0-1 encoding of the **labeled data**

$$\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N \Rightarrow \mathcal{D} = \{(\mathbf{x}_n, t_n)\}_{n=1}^N; \quad t_n = \begin{cases} 0 & \text{if } y_n = C_0 \\ 1 & \text{if } y_n = C_1 \end{cases}$$

$$\mathcal{L}: \mathcal{D} = \{(\mathbf{x}_n, c_n)\}_{n=1}^N \to o(\,\cdot\,; \mathbf{w})$$

$$z(\,\cdot\,; \mathbf{w}): \quad \mathbf{x} \in \mathcal{X} \to z(\mathbf{x}; \mathbf{w}) \in \mathbb{R}$$

$$o(\mathbf{x}; \mathbf{w}) = \varphi^{(o)}\big(z(\mathbf{x}; \mathbf{w})\big) \in [0,1]$$

- The output $o(\mathbf{x}; \mathbf{w})$ is an estimate of the class C_1 posterior

Discriminative model

$$\hat{p}(C_1 | \mathbf{x}, \mathbf{w}) = \varphi^{(o)}\big(z(\mathbf{x}; \mathbf{w})\big)$$

Logistic regression: $\varphi^{(o)}(z) = \sigma(z) = \frac{1}{1+e^{-z}}$

Probit regression: $\varphi^{(o)}(z) = \Phi(z) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{z} e^{-\frac{y^2}{2}} dy$

Binary classification: 0-1 encoding

n	x_{n1}	x_{n2}	...	x_{nD}	t_n
1	2.3	0	...	10.3	0
2	2.5	1	...	13.1	1
3	2.6	0	...	-2.7	1
4	2.7	-1	...	-5.4	0
5	2.9	0	...	2.1	1
6	3.1	0	...	-10.9	0

Learning by maximum lihelihood

Samples assumed to be iid (independent identically distributed)

$$\mathbf{w}^* = \arg\max_{\mathbf{w}} \mathcal{L}(\mathbf{w})$$

- Likelihood of the model, given $\mathcal{D} = \{(\mathbf{x}_n, t_n)\}_{n=1}^N$

$$\mathcal{L}(\mathbf{w}) = \hat{P}(\{t_n\}_{n=1}^N | \{\mathbf{x}_n\}_{n=1}^N, \mathbf{w}) = \prod_{n=1}^N \hat{p}(t_n | \mathbf{x}_n, \mathbf{w}) =$$

$$= \prod_{n=1}^N \left(\hat{p}(C_0 | \mathbf{x}_n, \mathbf{w})\right)^{1-t_n} \left(\hat{p}(C_1 | \mathbf{x}_n, \mathbf{w})\right)^{t_n}$$

Factorizes because samples are assumed to be independent

Estimate of class posterior

Same distribution: samples are assumed to be identically dist.

- Log-likelihood function

$$\log \mathcal{L}(\mathbf{w}) = \sum_{n=1}^N \left((1 - t_n) \log \hat{p}(C_0 | \mathbf{x}_n, \mathbf{w}) + t_n \log \hat{p}(C_1 | \mathbf{x}_n, \mathbf{w})\right)$$

Same maximizer as $\mathcal{L}(\mathbf{w})$ (log is monotone)

Minimization of the cross-entropy error

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} CE(\mathbf{w})$$

Same as maximizer of $\mathcal{L}(\mathbf{w})$

- Cross-entropy error

$$CE(\mathbf{w}) = -\log \mathcal{L}(\mathbf{w})$$

$$= -\sum_{n=1}^{N} \left((1-t_n)\log \hat{p}(C_0|\mathbf{x}_n,\mathbf{w}) + t_n \log \hat{p}(C_1|\mathbf{x}_n,\mathbf{w})\right)$$

$$= -\sum_{n=1}^{N} \left((1-t_n)\log\left(1-\hat{p}(C_1|\mathbf{x}_n,\mathbf{w})\right) + t_n \log \hat{p}(C_1|\mathbf{x}_n,\mathbf{w})\right)$$

- Discriminative model: $\hat{p}(C_1|\mathbf{x},\mathbf{w}) = \varphi^{(o)}\left(z(\mathbf{x}_n;\mathbf{w})\right)$

$\hat{p}(C_1|\mathbf{x},\mathbf{w})$

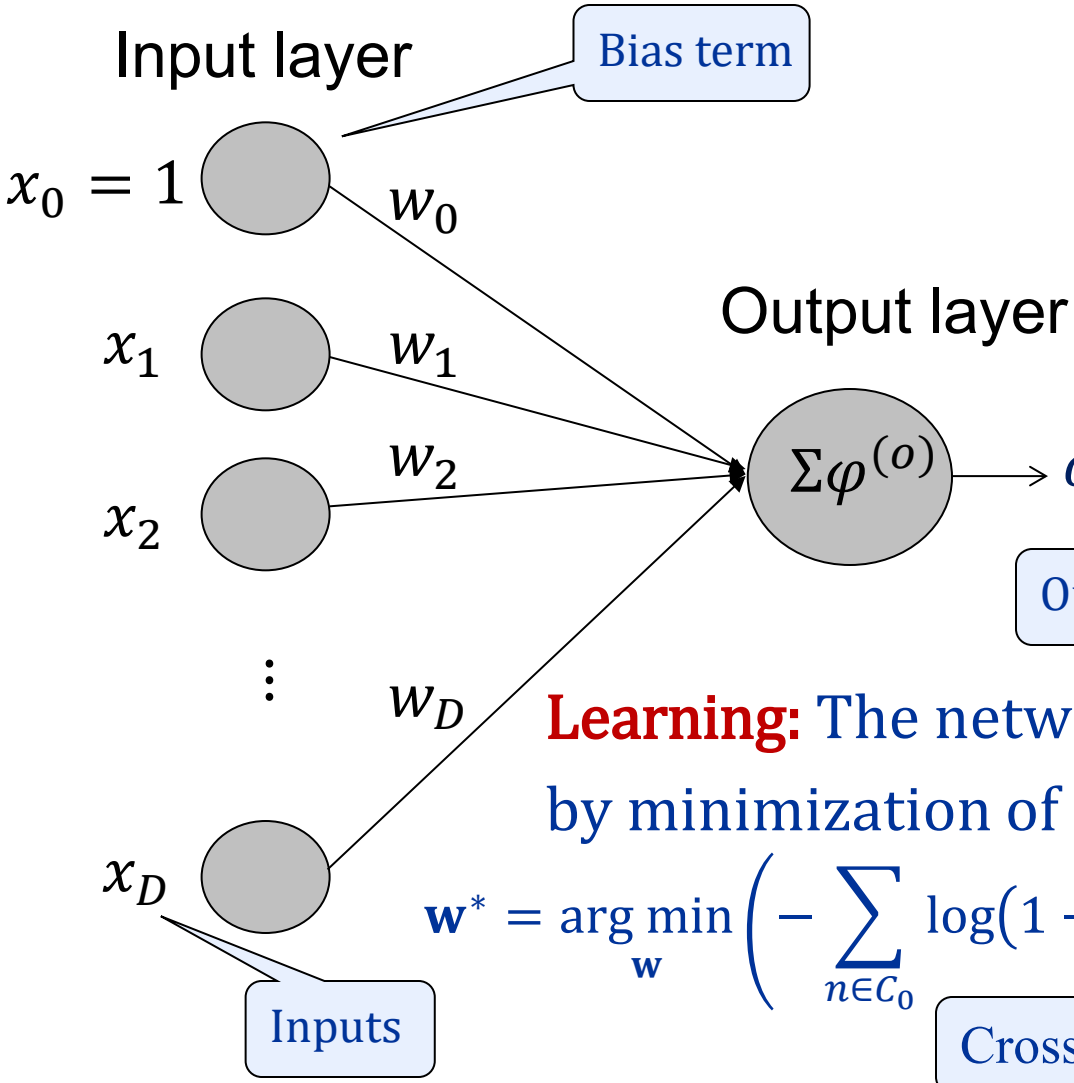
$$CE(\mathbf{w}) = -\sum_{n=1}^{N} \left((1-t_n)\log\left(1-\varphi^{(o)}\left(z(\mathbf{x}_n;\mathbf{w})\right)\right) + t_n \log\left(\varphi^{(o)}\left(z(\mathbf{x}_n;\mathbf{w})\right)\right)\right)$$

$\hat{p}(C_0|\mathbf{x},\mathbf{w})$

Single layer perceptron

Can solve only linearly separable problems

$$\mathbf{w}^{\mathrm{T}} = (w_0, w_1, \dots, w_D)$$
$$\mathbf{x}^{\mathrm{T}} = (x_0\ x_1 \dots x_D)$$
$$\mathbf{w}^{\mathrm{T}}\mathbf{x} = \sum_{d=0}^{D} w_d x_d$$



$$o(\mathbf{x}; \mathbf{w}) = \varphi^{(0)}\left(\mathbf{w}^{\mathrm{T}}\mathbf{x}\right)$$

Output is an estimate of $p(C_1|\mathbf{x})$

Learning: The network weights are determined by minimization of a cost function. For instance,

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \left(-\sum_{n \in C_0} \log\left(1 - o(\mathbf{x}_n; \mathbf{w})\right) - \sum_{n \in C_1} \log\left(o(\mathbf{x}_n; \mathbf{w})\right) \right)$$

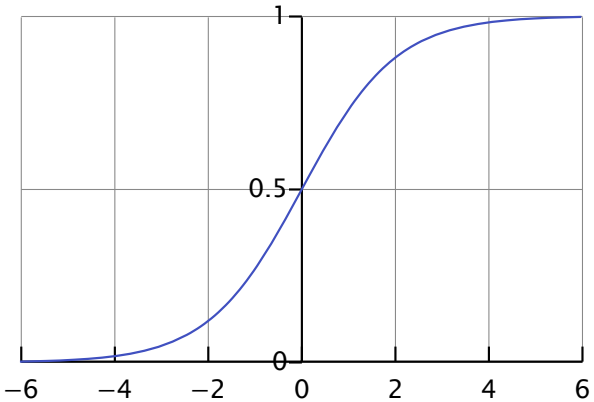
Cross-entropy error

The logistic sigmoid function

- **The logistic sigmoid function** :
  From real numbers to probabilities

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

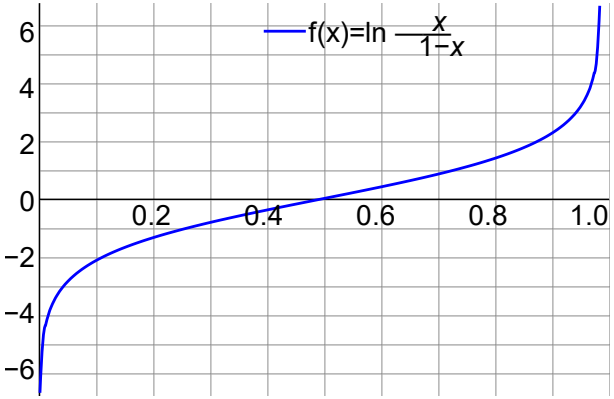
σ: $\mathbb{R} \longrightarrow [0, 1]$



- **The logit function**:
  From probabilities to real numbers

$$logit(p) = \sigma^{-1}(p) = \log\frac{p}{1 - p}$$

$logit$: $[0, 1] \longrightarrow \mathbb{R}$



File:Logistic-curve.svg. (2023, June 17). *Wikimedia Commons*. Retrieved 14:23, January 28, 2025 from https://commons.wikimedia.org/w/index.php?title=File:Logistic-curve.svg&oldid=775084440.

File:Logit.svg. (2022, February 22). *Wikimedia Commons*. Retrieved 14:25, January 28, 2025 from https://commons.wikimedia.org/w/index.php?title=File:Logit.svg&oldid=631647491.

Logistic regression

- Class posteriors

$$p(C_1|\mathbf{x},\mathbf{w}) = \frac{1}{1+e^{-z(\mathbf{x};\mathbf{w})}}$$

$$p(C_0|\mathbf{x},\mathbf{w}) = 1 - p(C_1|\mathbf{x},\mathbf{w}) = \frac{e^{-z(\mathbf{x};\mathbf{w})}}{1+e^{-z(\mathbf{x};\mathbf{w})}} = \frac{1}{1+e^{z(\mathbf{x};\mathbf{w})}}$$

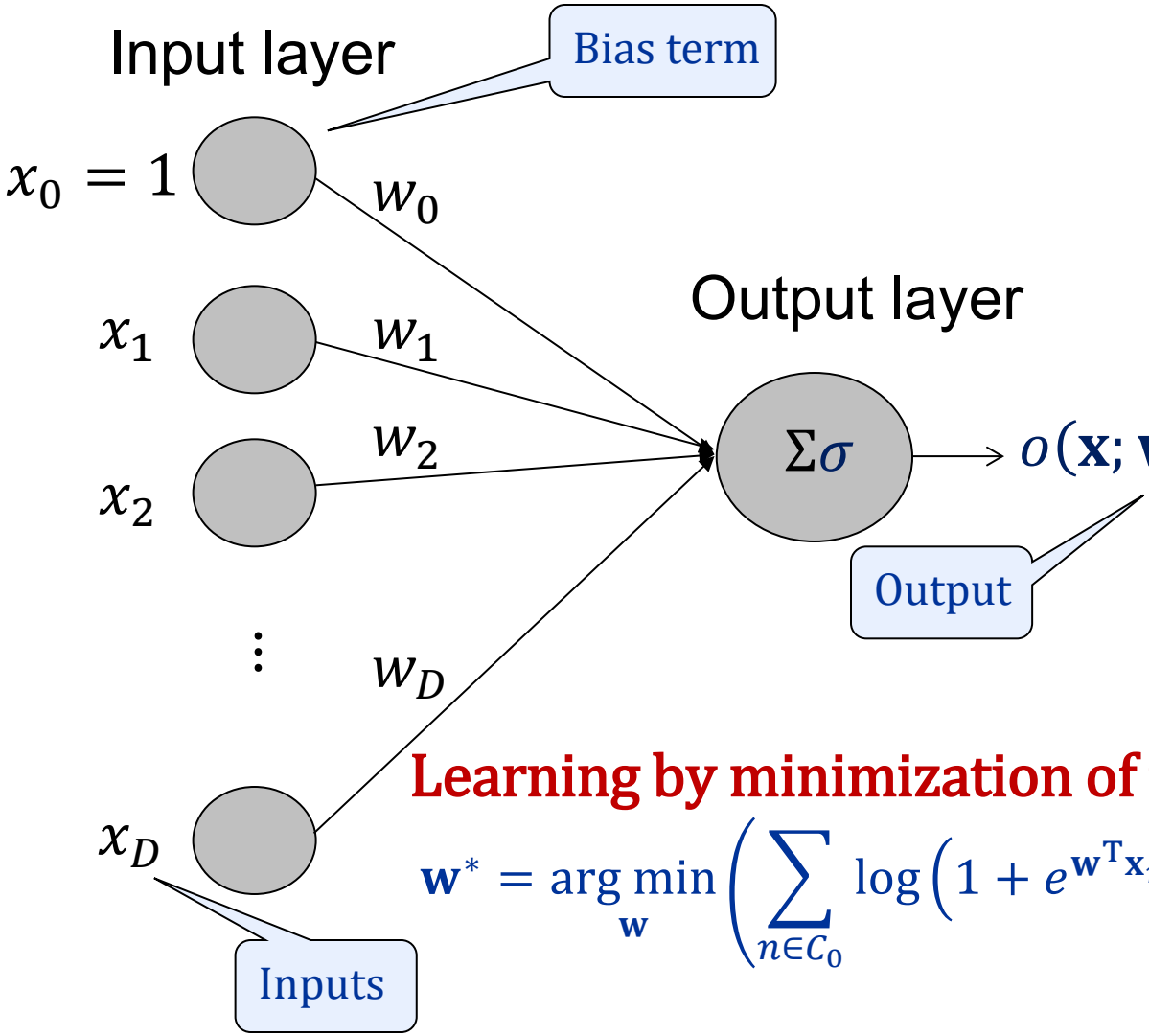
$\mathbf{w}^{\mathrm{T}} = (w_0, w_1, \dots, w_D)$
$\mathbf{x}^{\mathrm{T}} = (x_0\ x_1 \dots x_D)$

- Log-odds: $\mathrm{z}(\mathbf{x};\mathbf{w}) = \log\frac{p(C_1|\mathbf{x},\mathbf{w})}{p(C_0|\mathbf{x},\mathbf{w})}$

- Linear model for log-odds: $z(\mathbf{x};\mathbf{w}) = \mathbf{w}^{\mathrm{T}}\mathbf{x} = \sum_{d=0}^{D} w_d x_d$

Single-layer perceptron!

Single layer perceptron: logistic regression



$$\mathbf{w}^{\mathrm{T}} = (w_0, w_1, \dots, w_D)$$
$$\mathbf{x}^{\mathrm{T}} = (x_0\ x_1 \dots x_D)$$
$$\mathbf{w}^{\mathrm{T}}\mathbf{x} = \sum_{d=0}^{D} w_d x_d$$

$$o(\mathbf{x}; \mathbf{w}) = \sigma\left(\mathbf{w}^{\mathrm{T}}\mathbf{x}\right)$$

Learning by minimization of the cross-entropy error:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \left(\sum_{n \in C_0} \log\left(1 + e^{\mathbf{w}^{\mathrm{T}}\mathbf{x}_n}\right) + \sum_{n \in C_1} \log\left(1 + e^{-\mathbf{w}^{\mathrm{T}}\mathbf{x}_n}\right) \right)$$

Cross entropy error for logistic regression

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} CE(\mathbf{w})$$

$$\frac{\partial \sigma(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n)}{\partial \mathbf{w}} = \mathbf{x}_n \sigma(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n)\left(1 - \sigma(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n)\right)$$

- Cross-entropy error

$$CE(\mathbf{w}) = -\sum_{n=1}^{N}\left((1-t_n)\log\left(1-\sigma(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n)\right) + t_n \log \sigma(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n)\right)$$

- Gradient of the cross-entropy error

$$\frac{\partial}{\partial \mathbf{w}} CE(\mathbf{w}) = \sum_{n=1}^{N}\left((1-t_n)\mathbf{x}_n \sigma(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n) - t_n \mathbf{x}_n\left(1-\sigma(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n)\right)\right)$$

$$\delta_n = \sigma(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n) - t_n$$

$$= \sum_{n=1}^{N}\left(\sigma(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n) - t_n\right)\mathbf{x}_n = \sum_{n=1}^{N} \delta_n \mathbf{x}_n$$

Single-layer perceptron: Batch learning

INPUT: Training instances: $\mathcal{D} = \{(\mathbf{x}_n, t_n)\}_{n=1}^{N}$

Learning rate: $\eta > 0$

OUTPUT: $\mathbf{w}^* = \arg\min_{\mathbf{w}} CE(\mathbf{w})$

Attributes should be scaled!

1. Randomly initialize $\mathbf{w} \sim U[-0.5, 0.5]^{(D+1)}$
2. $n_{epoch} = 0$
2. While convergence criteria are not met

Similar to Rosenblatt!
$\delta_n = \sigma(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n) - t_n$
prediction error

2.1 Increment epoch counter: $n_{epoch} = n_{epoch} + 1$

2.2 Calculate the network outputs: $\sigma(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n)$; $n = 1, \ldots, N$

2.2 Calculate the gradient: $\frac{\partial}{\partial \mathbf{w}} CE(\mathbf{w}) = \sum_{n=1}^{N} \delta_n \, \mathbf{x}_n$

2.3 Update weights: $\mathbf{w} = \mathbf{w} - \eta \sum_{n=1}^{N} \delta_n \, \mathbf{x}_n$

Single-layer perceptron: Online learning

INPUT: Training instances: $\mathcal{D} = \{(\mathbf{x}_n, t_n)\}_{n=1}^{N}$

Learning rate: $\eta > 0$

OUTPUT: $\mathbf{w}^* = \arg\min_{\mathbf{w}} CE(\mathbf{w})$

1. Randomly initialize $\mathbf{w} \sim U[-0.5, 0.5]^{(D+1)}$
2. $n_{epoch} = 0$
3. While convergence criteria are not met

    3.1 Increment epoch counter: $n_{epoch} = n_{epoch} + 1$

    3.2 For $n = 1, \ldots, N$

    Calculate the network output: $\sigma\left(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n\right)$

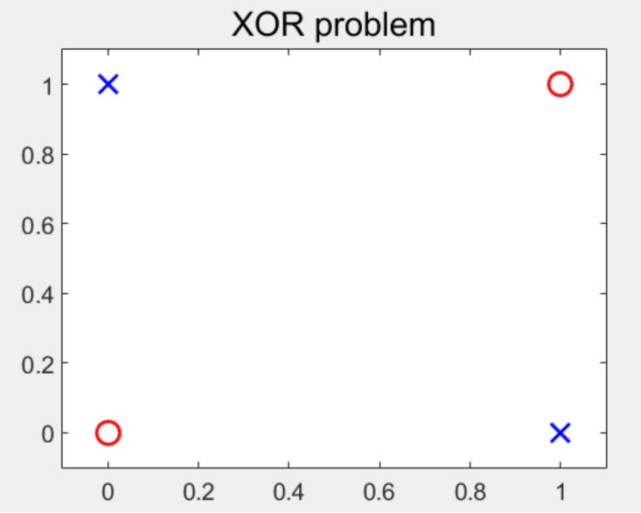
    Calculate prediction error: $\delta_n = \sigma\left(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n\right) - t_n$

    Update weights: $\mathbf{w} = \mathbf{w} - \eta\delta_n\,\mathbf{x}_n$

XOR: A non-linearly separable problem

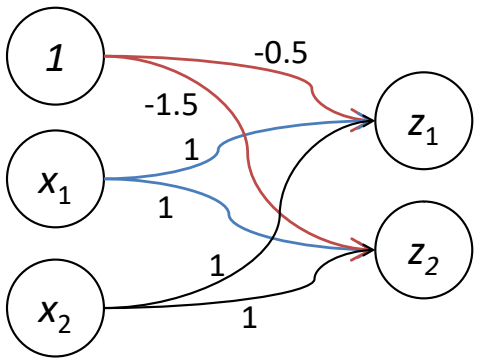
- The single-layer perceptron can only address problems that are linearly separable
- Therefore, the simple XOR problem, which is not linearly separable, cannot be solved using this learning machine.

x_1	x_2	h
0	0	1
0	1	0
1	0	0
1	1	1

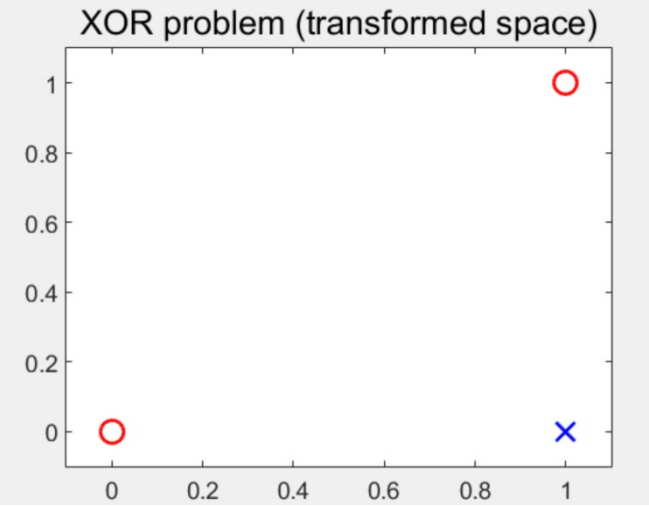


XOR: Non-linear feature construction

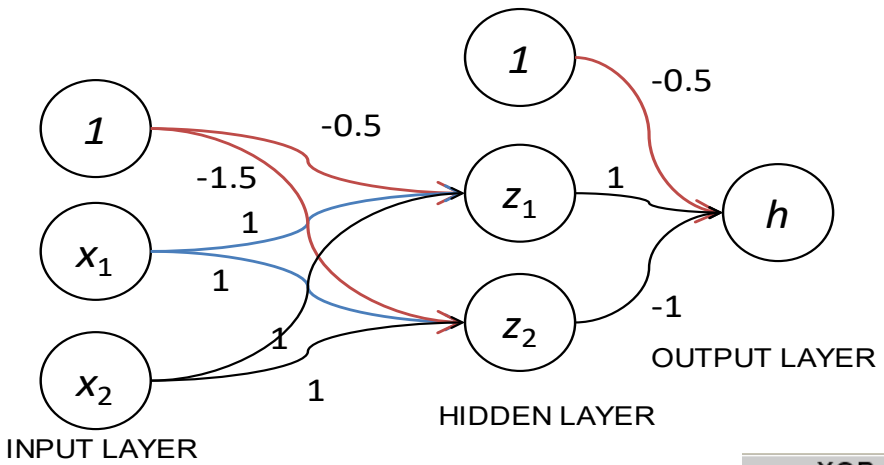
- Consider the XOR problem in a transformed feature space



x_1	x_2	z_1	z_2	h
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0



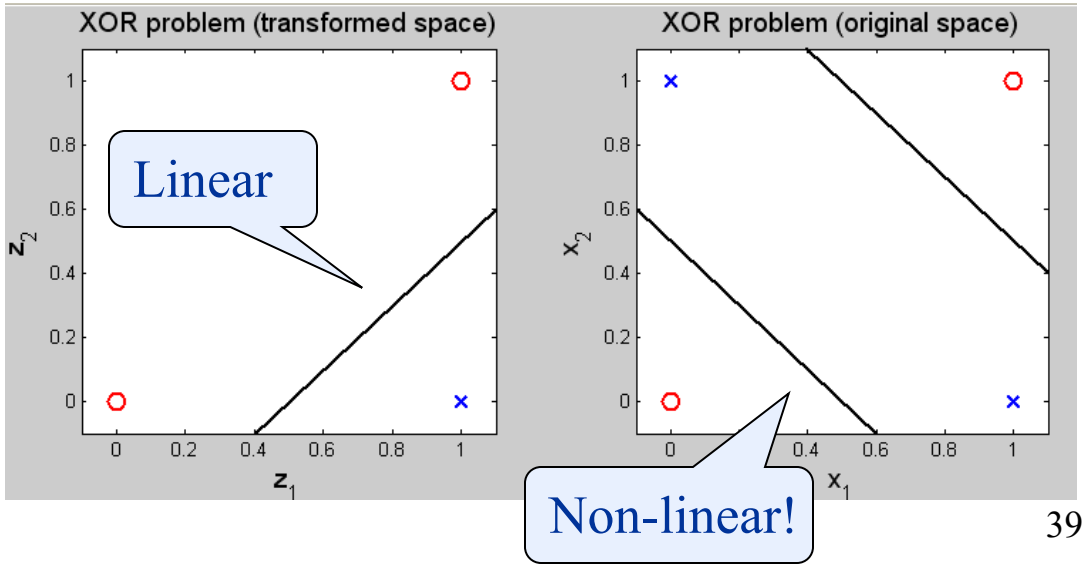
XOR: A linear model in feature space



x_1	x_2	z_1	z_2	h
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

$$\begin{cases} z_1 = \theta(x_1 + x_2 - 0.5) \\ z_2 = \theta(x_1 + x_2 - 1.5) \end{cases}$$

$$h = \theta(z_1 - z_2 - 0.5)$$



Multi-layer perceptron

$$\mathbf{x} = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_D \end{pmatrix}$$

Attribute (input) vector

input to hidden layer weights

$$\mathbf{V} = (\mathbf{v}_1 \ \mathbf{v}_2 \dots \mathbf{v}_J)$$

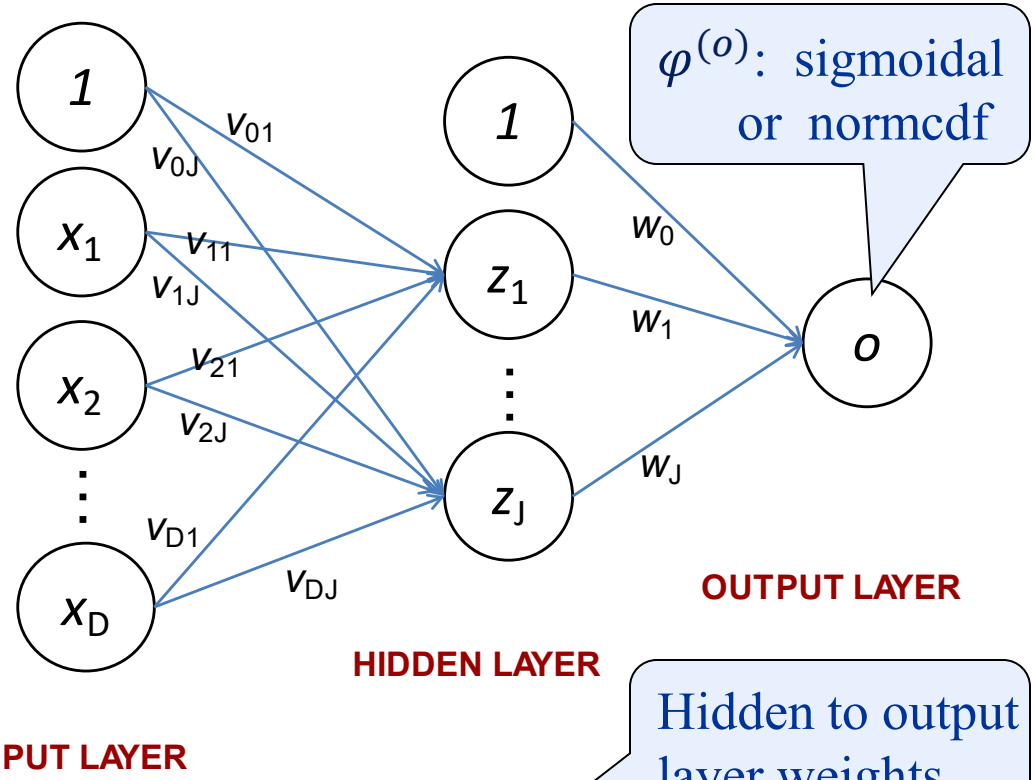
$$\mathbf{v}_j^{\mathrm{T}} = (\mathrm{v}_{0j} \ \mathrm{v}_{1j} \dots \mathrm{v}_{Dj});$$

$$\mathbf{z} = \begin{pmatrix} 1 \\ z_1 \\ \vdots \\ z_J \end{pmatrix};$$

Local field

$$z_j = \phi_j^{(H)}(\mathbf{v}_j^{\mathrm{T}}\mathbf{x}); \qquad j = 1, 2, \dots, J;$$

Activation function



Hidden to output layer weights

$$\mathbf{w}^{\mathrm{T}} = (\mathrm{w}_0 \ \mathrm{w}_1 \dots \mathrm{w}_D);$$

$$o(\mathbf{x}; \mathbf{w}) = \varphi^{(o)}(\mathbf{w}^{\mathrm{T}}\mathbf{z}) = \varphi^{(o)}\left(\sum_{j=0}^{J} w_j \, \phi_j^{(H)}(\mathbf{v}_j^{\mathrm{T}}\mathbf{x})\right)$$

Sigmoid (logit) activation function

Also: transfer function

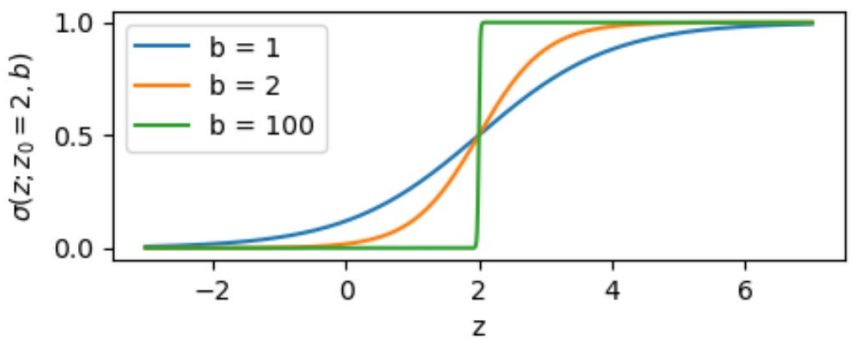
$$\sigma(z) = \frac{1}{1+e^{-z}};$$

- $\sigma(-\infty) = 0; \quad \sigma(0) = \frac{1}{2}; \sigma(+\infty) = 1;$
- Monotonically increasing: $z_2 > z_1 \Rightarrow \sigma(z_2) > \sigma(z_1)$
- Symmetry: $\sigma(-z) = 1 - \sigma(z)$
- Derivative: $\sigma'(z) = \frac{d\sigma(z)}{dz} = \sigma(z)\big(1-\sigma(z)\big)$

$$\sigma(z; z_0, b) = \frac{1}{1+e^{-b(z-z_0)}}$$

$\frac{1}{b}$: scale

z_0: center



$$\lim_{b\to\infty} \sigma(z; z_0, b) = \begin{cases} 0 & \text{if } z < z_0 \\ \frac{1}{2} & \text{if } z = z_0 \\ 1 & \text{if } z > z_0 \end{cases}$$

Heavyside step function

Other activation functions

In regression output layer

- **Linear**: $\varphi(z) = z$
- **Hyperbolic tangent**: $\tanh(z) = \frac{e^{bz} - e^{-bz}}{e^{bz} + e^{-bz}}$

normcdf

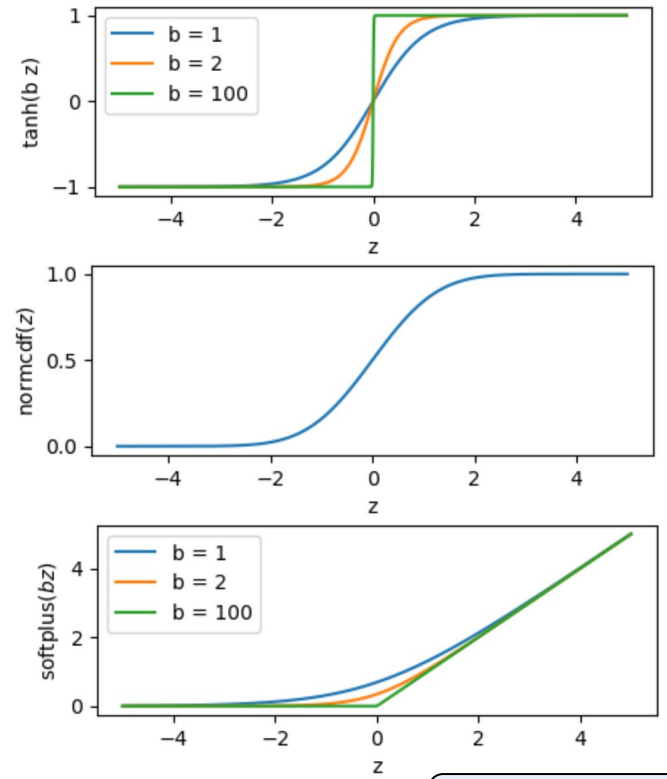
- **Probit**: $\Phi(z) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{z} e^{-\frac{y^2}{2}} dy$
- **softplus**: $\text{softplus}(bz) = \frac{log(1+e^{bz})}{b}$

$$\lim_{b \to \infty} \text{softplus}(bz) = \text{ReLU}(z)$$

- **Rectified linear unit**: $\text{ReLU}(z) = \max(0, z)$
- **Leaky rectified linear unit**: $\text{Leaky ReLU}(z) = \begin{cases} az & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}; \quad 0 < a \ll 1$

Avoids zero gradient if $z < 0$

E.g. $a = 0.01$



Multi-layer perceptron: Online learning

INPUT: Training instances: $\mathcal{D} = \{(\mathbf{x}_n, t_n)\}_{n=1}^{N}$

Learning parameter: $\eta > 0$

OUTPUT: $\mathbf{V}^*, \mathbf{w}^* = \arg\min_{\mathbf{V},\mathbf{w}} CE(\mathbf{V}, \mathbf{w})$

Only one hidden layer.
Sigmoid activations.

1. Randomly initialize $\mathbf{V}, \mathbf{w} \sim U[-0.5, 0.5]^{(D+1)}$
2. $n_{epoch} = 0$
3. While convergence criteria are not met

   3.1 Increment epoch counter: $n_{epoch} = n_{epoch} + 1$

   3.2 For $n = 1, \dots, N$

   $z_{nj} = \sigma(\mathbf{v}_j^{\mathrm{T}} \mathbf{x}_n), \quad j = 1, 2, \dots, J;$ **# forward propagation**

   $o_n = \sigma(\mathbf{w}^{\mathrm{T}} \mathbf{z}_n)$ # network output

   $\delta_n = o_n - t_n;$ # prediction error

   $\mathbf{w} = \mathbf{w} - \eta \delta_n \mathbf{z}_n$ # weight update

   $\Delta_{nj} = z_{nj}(1 - z_{nj})\delta_n, \quad j = 1, 2, \dots, J;$ **# error backpropagation**

   $\mathbf{v}_j = \mathbf{v}_j - \eta \Delta_{nj} \mathbf{x}_n$ # weight update

Universal approximation property

THEOREM: "A feed-forward network with a **single hidden layer** containing a **sufficiently large number of hidden neurons** can uniformly **approximate any continuous function** on compact subsets of $\mathbb{R}^D$, under mild conditions on the activation function"

- GOOD NEWS:

  A neural network can be used to make optimal predictions!

- NOT SO GOOD NEWS:

  How does one find such a network?

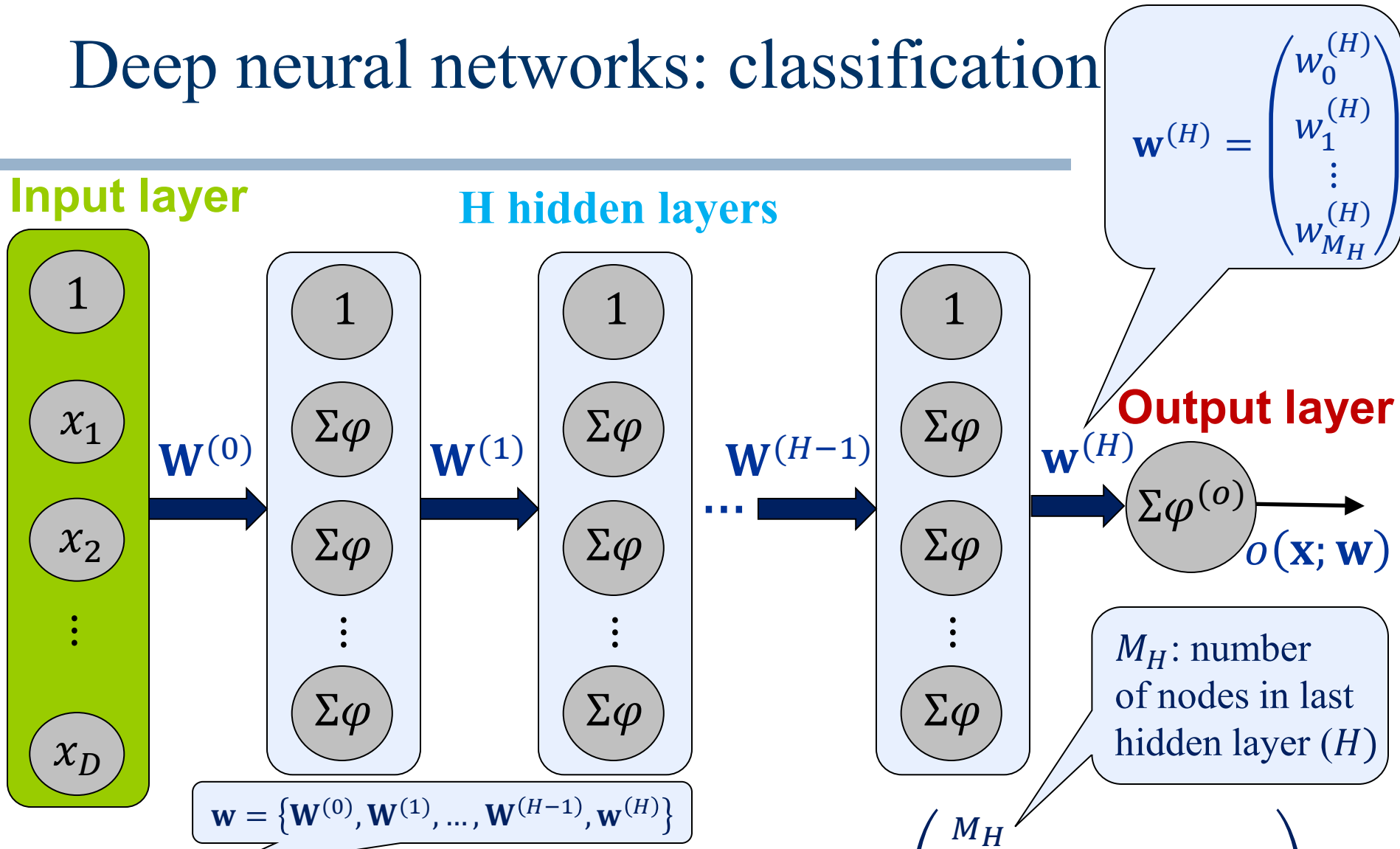
  - How many neurons do we need?
  - How easy is it to learn the network parameters (weights)?

Use CV

Having more than one layer can help (deep networks)

K. Hornik, M. Stinchcombe and H. White, "Multi-layer Feedforward Networks are Universal Approximators," Neural Networks, Vol. 2, pp. 359-366 (1989).

Deep neural networks: classification

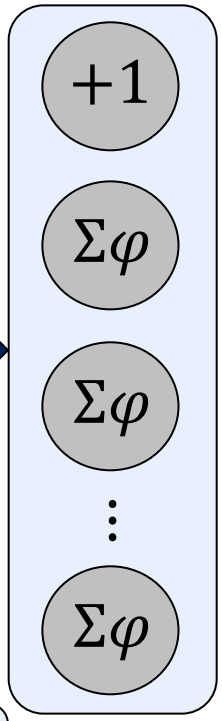


$$o(\mathbf{x};\mathbf{w}) = \varphi^{(o)}\left(\left(\mathbf{w}^{(H)}\right)^T \boldsymbol{\phi}^{(H)}(\mathbf{x})\right) = \varphi^{(o)}\left(\sum_{m=0}^{M_H} w_m^{(H)} \phi_m^{(H)}(\mathbf{x})\right)$$

Output of hidden layer h

$$\mathbf{W}^{(h-1)} = \left(\mathbf{w}_1^{(h-1)} \mathbf{w}_2^{(h-1)} \dots \mathbf{w}_{M_h}^{(h-1)}\right)$$

$$\boldsymbol{\phi}^{(h-1)}(\mathbf{x}) \xrightarrow{\mathbf{W}^{(h-1)}}$$



$$\boldsymbol{\phi}^{(h)}(\mathbf{x}) = \begin{pmatrix} 1 \\ \phi_1^{(h)}(\mathbf{x}) \\ \vdots \\ \phi_{M_h}^{(h)}(\mathbf{x}) \end{pmatrix}$$

$$\phi_0^{(h)} = 1$$

$$\boldsymbol{\phi}^{(0)}(\mathbf{x}) = \mathbf{x} = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_D \end{pmatrix}$$

$$M_0 = D$$

$$\boldsymbol{\phi}^{(h-1)}(\mathbf{x}) = \begin{pmatrix} 1 \\ \phi_1^{(h-1)}(\mathbf{x}) \\ \vdots \\ \phi_{M_{h-1}}^{(h-1)}(\mathbf{x}) \end{pmatrix}$$

$$\left(\mathbf{w}_m^{(h-1)}\right)^{\mathrm{T}} = \left(w_{0m}^{(h-1)} \; w_{1m}^{(h-1)} \dots w_{M_{h-1}m}^{(h-1)}\right)$$

$$\phi_m^{(h)}(\mathbf{x}) = \varphi^{(h)}\left(\left(\mathbf{w}_m^{(h-1)}\right)^{\mathrm{T}} \boldsymbol{\phi}^{(h-1)}(\mathbf{x})\right); \; m = 1, \dots, M_h$$

$$M_{H+1} = 1$$

In the last layer: $\varphi^{(H+1)}(z) = \varphi^{(o)}(z)$ (logistic, probit,…)

$$h = 1, \dots, H, H+1$$

Learning the network weights

The network weights are determined by minimization of a cost function:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \left(\left[-\sum_{n \in C_0} \log\big(1 - o(\mathbf{x}_n; \mathbf{w})\big) - \sum_{n \in C_1} \log\big(o(\mathbf{x}_n; \mathbf{w})\big) \right] + \lambda_1 \|\mathbf{w}\|_1 + \lambda_2 \|\mathbf{w}\|_2^2 \right)$$

Cross-entropy error

$\lambda_1 > 0$ | L_1 penalty | $\lambda_2 > 0$ | L_2 penalty

Use CV to select

- **Architecture**: Number of hidden layers / Neurons in hidden layer
- **Optimization method**: Quasi-Newton, Gradient descent, stochastic gradient descent, use of momentum… Gradients needed!
- **Hyperparameters**: Magnitude of penalties, of momentum term, maximum # of iterations, …

Multi-class classification (K classes)

1-of K encoding of the **labeled data**

$$\mathcal{D} = \{(\mathbf{x}_n, c_n)\}_{n=1}^{N} \Rightarrow \mathcal{D} = \{(\mathbf{x}_n, \mathbf{t}_n)\}_{n=1}^{N}; \quad \boldsymbol{t}_n^{\mathrm{T}} = (t_{n1}\ t_{n2} \dots t_{nK})$$

$$t_{nk} = \mathbb{I}[c_n = C_k] = \begin{cases} 0 & \text{if} \quad y_n \neq C_k \\ 1 & \text{if} \quad y_n = C_k \end{cases}; \quad k = 1, 2, \dots, K$$

$$\mathcal{L}\colon \mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^{N} \to o(\,\cdot\,; \mathbf{w})$$

$$z(\,\cdot\,; \mathbf{w})\colon \quad \mathbf{x} \in \mathcal{X} \ \to \mathbf{z}(\mathbf{x}; \mathbf{w}) \in \mathbb{R}^K$$

$$\mathbf{o}(\mathbf{x}; \mathbf{w}) = \boldsymbol{\varphi}^{(\mathrm{o})}\big(\mathbf{z}(\mathbf{x}; \mathbf{w})\big) \in \Delta^K$$

Probability simplex in K dimensions

Discriminative model

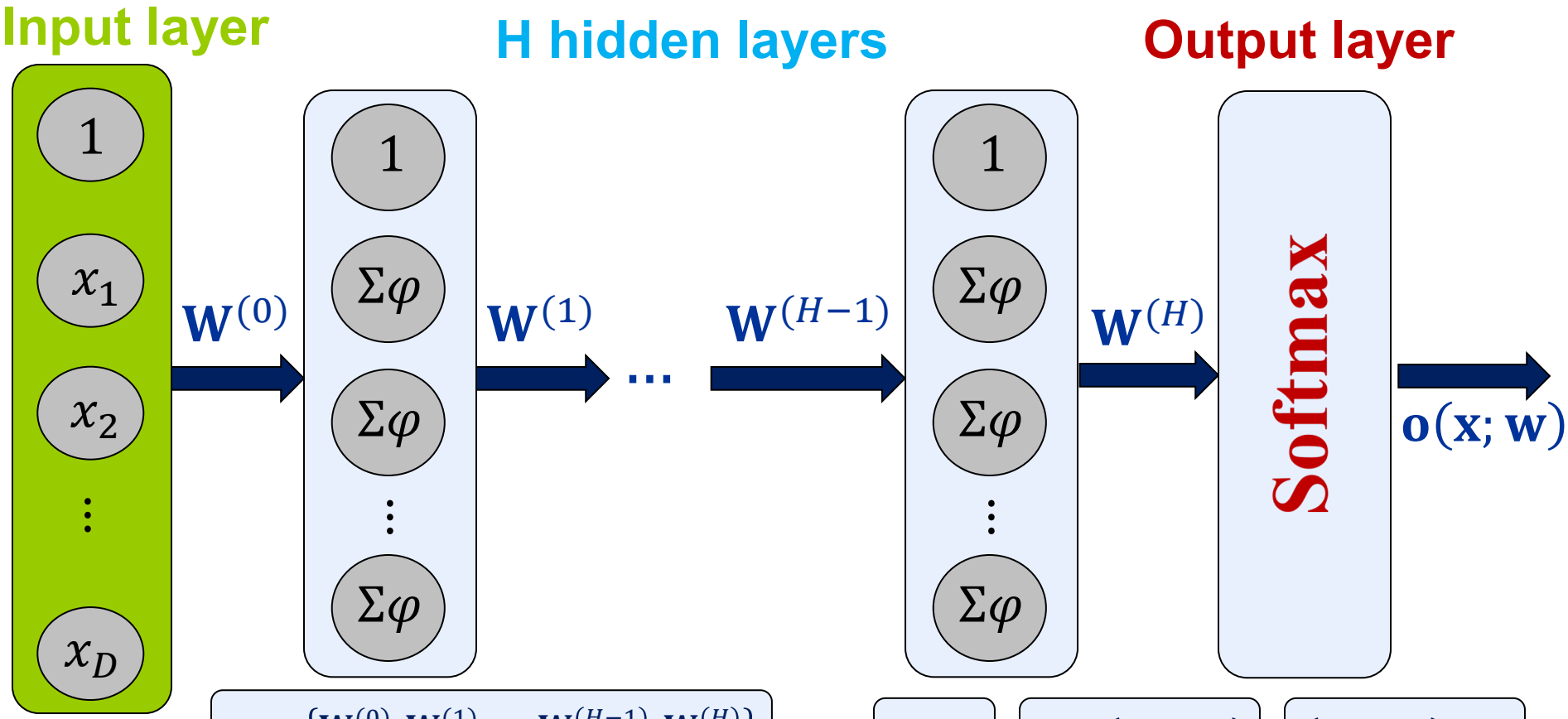
$\hat{p}(C_k|\mathbf{x}, \mathbf{w}) = \varphi_K^{(o)}\big(\mathbf{z}(\boldsymbol{x}; \mathbf{w})\big)$

$$\varphi_k^{(o)}(\mathbf{z}) \geq 0 \qquad k = 1, \dots, K$$

$$\varphi_1^{(o)}(\mathbf{z}) + \cdots + \varphi_K^{(o)}(\mathbf{z}) = 1$$

$$\left(\boldsymbol{\varphi}^{(\mathrm{o})}(\mathbf{z})\right)^{\mathrm{T}} = \left(\varphi_1^{(o)}(\mathbf{z}) \dots \varphi_K^{(o)}(\mathbf{z})\right)$$

MLP for multiclass classification



$\mathbf{w} = \{\mathbf{W}^{(0)}, \mathbf{W}^{(1)}, \dots, \mathbf{W}^{(H-1)}, \mathbf{W}^{(H)}\}$

$K \times 1$ $\quad$ $K \times (M_H + 1)$ $\quad$ $(M_H + 1) \times 1$

$K \times 1$

$$\mathbf{o}(\mathbf{x};\mathbf{w}) = \boldsymbol{\varphi}^{(o)}\big(\mathbf{z}(\mathbf{x},\mathbf{w})\big) = \boldsymbol{\varphi}^{(o)}\left(\left(\mathbf{W}^{(H)}\right)^T \boldsymbol{\phi}^{(H)}(\mathbf{x})\right)$$

Softmax layer

$$\mathbf{z}(\mathbf{x};\mathbf{w}) = \begin{pmatrix} z_1(\mathbf{x},\mathbf{w}) \\ z_2(\mathbf{x},\mathbf{w}) \\ \vdots \\ z_K(\mathbf{x},\mathbf{w}) \end{pmatrix} \rightarrow \boxed{\textbf{Softmax}} \rightarrow \boldsymbol{\varphi}^{(o)}\big(\mathbf{z}(\mathbf{x};\mathbf{w})\big) = \begin{pmatrix} \varphi_1^{(o)}\big(\mathbf{z}(\mathbf{x},\mathbf{w})\big) \\ \varphi_2^{(o)}\big(\mathbf{z}(\mathbf{x},\mathbf{w})\big) \\ \vdots \\ \varphi_K^{(o)}\big(\mathbf{z}(\mathbf{x},\mathbf{w})\big) \end{pmatrix}$$

$$\hat{p}(C_k|\mathbf{x},\mathbf{w}) = \varphi_k^{(o)}\big(\mathbf{z}(\mathbf{x},\mathbf{w})\big) \qquad k = 1,\dots,K$$

Probabilities

$$\varphi_k^{(o)}(\mathbf{z}) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \geq 0 \qquad k = 1,\dots,K$$

$$\varphi_1^{(o)}(\mathbf{z}) + \cdots + \varphi_K^{(o)}(\mathbf{z}) = 1$$

Learning by maximum lihelihood

Samples assumed to be iid (independent identically distributed)

$$\mathbf{w}^* = \arg\max_{\mathbf{w}} \mathcal{L}(\mathbf{w})$$

- Likelihood of the model, given $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{t}_n)\}_{n=1}^N$

$$\mathcal{L}(\mathbf{w}) = \hat{P}(\{\mathbf{t}_n\}_{n=1}^N | \{\mathbf{x}_n\}_{n=1}^N, \mathbf{w}) = \prod_{n=1}^N \hat{p}(t_n | \mathbf{x}_n, \mathbf{w}) =$$

$$= \prod_{n=1}^N \prod_{k=1}^K \left(\hat{p}(C_k | \mathbf{x}_n, \mathbf{w})\right)^{t_{nk}}$$

Factorizes because samples are assumed to be independent

Factors equal because samples are assumed to be identically dist.

- Log-likelihood function

$t_{nk} = \mathbb{I}[c_n = C_k]$

$$\log \mathcal{L}(\mathbf{w}) = \sum_{n=1}^N \sum_{k=1}^K t_{nk} \log \hat{p}(C_k | \mathbf{x}_n, \mathbf{w})$$

Minimization of the cross-entropy error

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} CE(\mathbf{w})$$

- Cross-entropy error

$$CE(\mathbf{w}) = -\log \mathcal{L}(\mathbf{w}) = -\sum_{n=1}^{N}\sum_{k=1}^{K} t_{nk} \log \hat{p}(C_k|\mathbf{x}_n, \mathbf{w})$$

- Discriminative model: $\hat{p}(C_1|\mathbf{x}, \mathbf{w}) = \varphi_k^{(o)}\big(z(\mathbf{x}_n, \mathbf{w})\big)$

Output of the kth neuron in the last layer of the MLP.

$$\widehat{CE}(\mathbf{w}) = -\sum_{n=1}^{N}\sum_{k=1}^{K} t_{nk} \log \varphi_k^{(o)}\big(z(\mathbf{x}_n, \mathbf{w})\big)$$

Neural networks summary

- **Advantages**
  - Excellent predictors.
  - Adaptive (online learning)
- **Disadvantages**
  - Costly training.
  - Finding appropriate architecture can be difficult
    - Number of hidden layers / nodes in each hidden layer
    - Activation function
  - Determining the hyperparameters for the optimization
    - Type of optimization
    - In SGD: learning rate, size of mini-batches, momentum term, strength of regularization terms, …
  - Difficult interpretation.

State-of-the-art: Deep neural networks

References

- Neural playground: Multilayer perceptron with TensorFlow: https://playground.tensorflow.org/
- Scikit learn documentation: http://scikit-learn.org/stable/modules/neural_networks_supervised.html
- Simon O. Haykin "Neural Networks and Learning Machines, 3rd edition", Pearson (2009)
- Ian Goodfellow and Yoshua Bengio and Aaron Courville "Deep Learning", MIT Press (2016)
- Digits: https://www.cs.ryerson.ca/~aharley/vis/conv/
- 3Blue1Brown
  - Part 1: https://www.youtube.com/watch?v=aircAruvnKk
  - Part 2: https://www.youtube.com/watch?v=IHZwWFHWa-w&t=331s
  - Part 3: https://www.youtube.com/watch?v=Ilg3gGewQ5U
  - Part 4: https://www.youtube.com/watch?v=tIeHLnjs5U8
- Deep neural networks: https://www.youtube.com/watch?v=0VH1Lim8gL8